

Tic Tac Toe (tkinter GUI)

Brief

Two Python files. No third-party dependencies. An opponent that cannot be beaten, and a script in the repository that proves it.

1 The pitch in one page

A Tic Tac Toe game in a real window. Click a square, your mark appears, the program decides whether anyone has won. Two modes: two people at one keyboard, or one person against a computer that always wins or draws and never loses.

That last claim is the whole point, and it is the reason the project is worth a document. “Unbeatable” is normally marketing. Here it is a property, and the repository contains the check that would catch it being false.

The single decision everything else follows from

The rules live in `game.py`. The window lives in `main.py`. The arrow points one way: `main.py` imports `game.py`, and `game.py` imports nothing but `math`. It has never heard of a screen. That is why a script with no display attached can play out every game the AI can ever face. If the rules had been written inside the button handler, as they easily could have been at this size, the “unbeatable” claim would have been untestable and therefore just words.

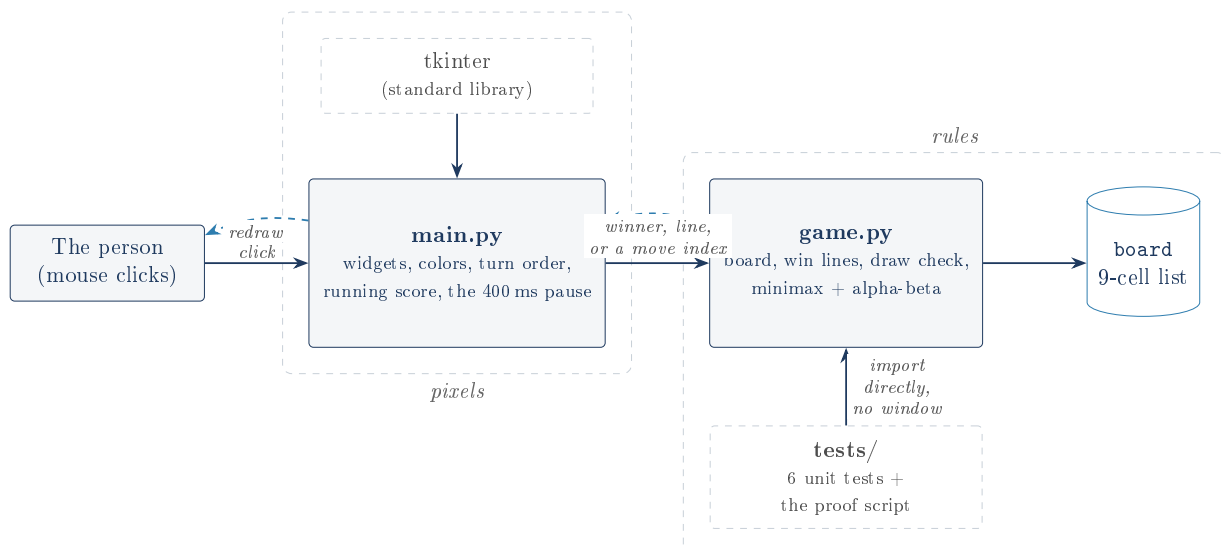


Figure 1: The dependency arrow never points back. The tests reach past the window entirely, which is the property the project is built around.

2 The rules layer, and the bug that shaped it

Per the README, this started as a console practice script with a real defect: its win check only ever compared cells against " X ", so player O could complete a line and the game carried on regardless. It was written as eight `or`-chained comparisons, and one missing case in eight near-identical lines is exactly the kind of thing nobody sees.

The rewrite does not add the missing check. It stops writing the check per mark:

```

1 WIN_LINES = [
2     (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
3     (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
4     (0, 4, 8), (2, 4, 6),           # diagonals
5 ]
6
7 def check_winner(board):
8     for line in WIN_LINES:
9         a, b, c = line
10        if board[a] != EMPTY and board[a] == board[b] == board[c]:
11            return board[a], line
12    return None

```

The board is one flat list of nine strings, indexed 0 to 8 left to right and top to bottom, so every win line is three plain integers. The mark is never named in the comparison: the test is “these three cells match and are not empty”, which covers X and O in one code path. The bug becomes unrepresentable rather than merely fixed, and eight tuples can be checked by eye against a picture of a board in seconds, which eight branches cannot.

`check_winner` returns the winning *line* as well as the mark, because the GUI wants to paint those three cells gold. The rules layer anticipates that need but answers it with three integers, not a colour. Around it sit six more small total functions: `new_board`, `make_move` (rejects an occupied or out-of-range cell by returning `False`), `is_draw` (full board *and* no winner), `is_game_over`, `available_moves` and `other_mark`.

3 The AI

Tic Tac Toe is small. From an empty board there are at most $9! = 362,880$ move orderings, and a computer can look at all of them faster than a human blinks. So the AI does not use heuristics like “take the centre”. It brute-forces the question.

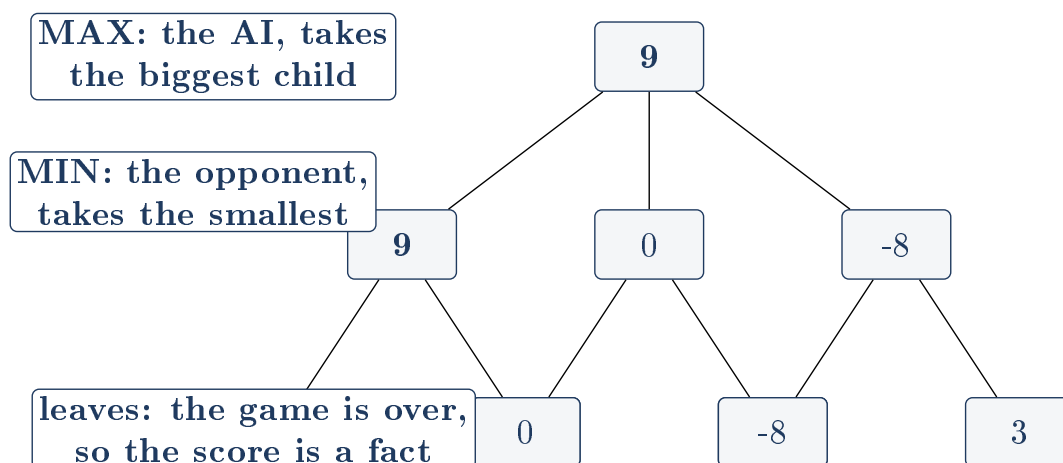


Figure 2: Minimax. Leaf scores are always from the AI’s point of view: a win is positive, a loss negative, a draw zero. The AI assumes perfect defence at every MIN node and still finds a line it is happy with, which is why the guarantee holds against *any* opponent rather than a merely bad one.

At a leaf, `_score_terminal` returns `10 - depth` for an AI win and `depth - 10` for a loss, where `depth` counts moves from the current board. The depth term is a tie-break: two winning paths, and the AI takes the shorter one, so it closes out a won game instead of shuffling. `best_ai_move` is the entire public surface, four lines wrapping the search into “give me a board and a mark, get back an index”. The GUI has no idea a tree search happened.

Alpha-beta pruning skips branches that cannot change the answer: if you have already secured a 5 elsewhere and this branch has already dropped to 3, the opponent will send you to the 3, so

the rest of the branch is unreachable.

Pruning is not why the AI is unbeatable

This is the distinction most write-ups blur, and the README names correcting it as one of the project's actual challenges. The guarantee comes *entirely* from exhaustive minimax search. Delete alpha and beta and the AI is exactly as unbeatable, only slower. Bolt alpha-beta onto a broken search and you get a fast broken search. Pruning changes how much of an already-decided tree gets visited; it never changes the decision. Merging the two claims would quietly overstate what the optimisation contributes.

The AI is fully deterministic: no randomness, ties broken by board order. On an empty board it therefore always opens at index 0, the top-left corner.

4 The window

`main.py` is one class, `TicTacToeApp`. It builds a status label, two mode radio buttons, a mark chooser that is hidden until AI mode is selected, nine gridded buttons, “Play again”, and a score line. It holds the board, whose turn it is, two booleans (`game_over`, `ai_thinking`) and two score dictionaries. It holds no rules. The AI's mark is not even stored; it is a property computing `other_mark(self.human_mark.get())` on every read, so it cannot drift out of sync with the radio button.

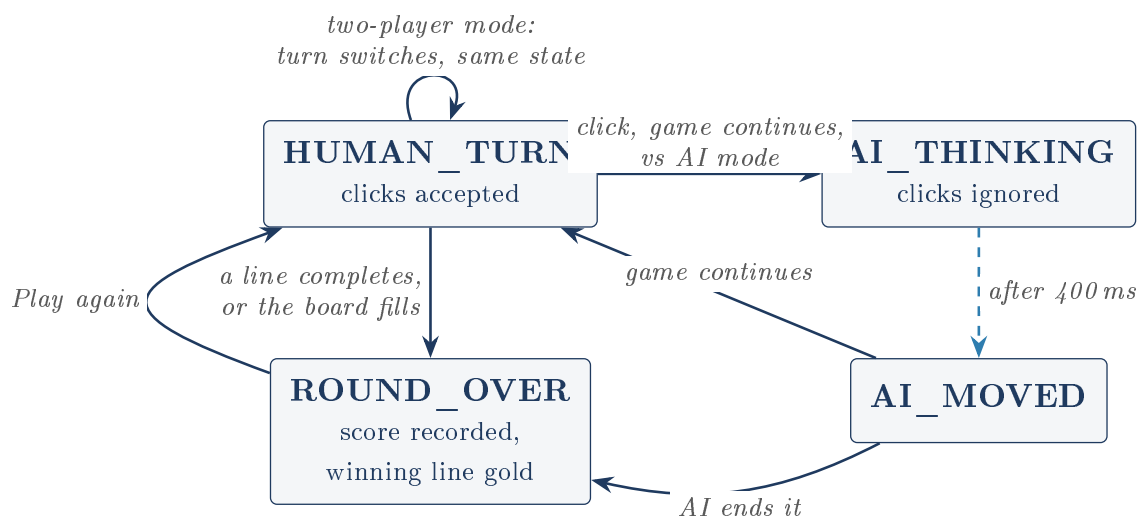


Figure 3: One round. In two-player mode only the self-loop is ever used.

Three points of substance:

Every click passes three guards. Round over or AI pending; the AI's turn in AI mode; and the cell already taken. The third is delegated: the GUI does not inspect the board, it calls `make_move` and reads the boolean. The rules layer is the authority even on the trivial question.

The 400 ms pause is cosmetic, and correctly implemented. The move is computed in about 0.055 seconds, so it would otherwise appear instantly. The source comment is explicit that the delay exists “so the AI doesn't feel like it's snapping the board shut”. It uses `root.after`, not `time.sleep`: sleeping would block tkinter's event loop and freeze the whole window for 400 ms. The `ai_thinking` flag is what stops the human clicking during a window that stays deliberately alive.

Two score dictionaries, keyed differently, on purpose. Two-player scores are keyed by mark; AI-mode scores by role (`"human"` / `"ai"`). In AI mode the player can switch sides between

rounds, so a mark-keyed tally would scatter one person's wins across two counters. Different question, different key.

5 The proof, and I reran it

`tests/verify_unbeatable.py` attacks the claim three ways, each for both starting orders, and exits non-zero with the losing board if the AI loses once.

Why one of the three checks is worth more than the other two

Random play can only ever *fail to find* a counterexample; it samples, so a loss needing one exact sequence could hide from a million games. The exhaustive check does not sample. At every human turn it recurses into every legal move, so it visits every distinct game any human strategy whatsoever could produce against this AI. If a loss existed, a leaf of that traversal would be it. That is the gap between “we did not observe a loss” and “a loss does not exist”, and the README's own conclusion is exactly this: a claim is only worth writing down if something behind it would catch the claim being false.

Every figure the README states reproduces exactly on this machine, in 57 seconds:

```

1  === 1. Exhaustive search over every human move sequence ===
2    [human moves first] 569 distinct games explored -> AI won: 386, draws: 183, AI lost: 0
3    [AI moves first] 73 distinct games explored -> AI won: 71, draws: 2, AI lost: 0
4
5  === 2. 1000 games, human plays uniformly random legal moves ===
6    [human moves first] 1000 games -> AI won: 789, draws: 211, AI lost: 0
7    [AI moves first] 1000 games -> AI won: 996, draws: 4, AI lost: 0
8
9  === 3. Both sides play optimally (human also uses minimax) ===
10   [human moves first] outcome: draw
11   [AI moves first] outcome: draw
12
13  RESULT: the AI never lost a single game across all checks above.
```

Listing 1: `python3 tests/verify_unbeatable.py`

569 and 73 are not the number of Tic Tac Toe games. They are the number reachable against this *deterministic* opponent, which answers each position with exactly one move. Check 2 is seeded, so 789 and 211 are reproducible rather than typical. Check 3 ending in a draw both ways is the game-theoretically correct result for perfect play. Alongside it, `tests/test_ai.py` runs six unit tests on specific tactical positions; all six pass in 0.047 seconds.

6 Honest findings

The AI claim survives scrutiny: it is verified exhaustively and I reproduced the verification. The defects are all in the layer nobody tested, which is where one would predict them.

A reachable GUI bug: the AI can be made to move twice

`root.after` returns a timer id that `after_cancel` could use. It is discarded, and `after_cancel` appears nowhere in the file. So clicking “Play again” during the AI's 400 ms pause leaves a live timer that `reset_board` cannot stop. Play on and a second timer is scheduled while the first is still pending; both fire, and `_make_ai_move` re-checks only `game_over`, not whose turn it is, so the AI plays a mark out of turn. The fix is two lines: keep the id, cancel it in `reset_board`.

The discarded return value and the missing `after_cancel` are verified by `grep`. The resulting double move is **inferred**: traced through the control flow, not observed, because this machine has no display server and no `Xvfb` to drive the window with.

Six unit tests, five distinct cases

`test_prefers_win_over_block_when_both_available` builds the exact same board as `test_takes_immediate_winning_move` and makes the exact same assertion. The comment above it is a train of thought that never landed: it spots that the win cell and the block cell coincide, writes “use a layout where winning and blocking are genuinely different cells”, abandons a third idea with “so build a cleaner case instead:” and then rebuilds the original board. The behaviour the test is named after is covered by nothing. The comment flags the trap and the code walks into it. Verified by comparing the two literals.

A comment says the AI opens in the centre; it opens in a corner

`test_ai.py` claims a centre-first opening “is what this specific minimax tie-breaking ... should produce”. It produces index 0, because 0 is enumerated before 4 and both score 0. The comment half-catches itself and the assertion is loosened to accept any corner or the centre, so the test is green and its prose is wrong. Both openings are game-theoretically fine, so nothing is broken; the reasoning is just stale. Verified by calling the function.

Nothing tests `main.py` at all

The verification exercises `game.py` only. Every GUI claim in this document is read, not run. That is an honest allocation of effort, since the interesting logic is all in the other file, but it does mean the timer bug above could never have been caught by the suite as it stands.

Smaller things, stated plainly

`docs/screenshots/app.png` shows a build with no mode radio buttons and no mark chooser, which the current `_build_layout` cannot produce; it predates the AI mode (verified by viewing it, and it is not referenced from the README, so no false claim is made). `_finish_turn_if_over` takes a `mark_just_played` argument that its body never reads, and both call sites dutifully pass one. `make_move` will accept any string as a mark, and `other_mark` has no third branch; neither is reachable from the real callers. The 10 in `_score_terminal` must exceed the maximum depth of 9 or wins would read as losses, and nothing in the code, the comments or the README says so (that constraint is my reading of the arithmetic, so: **inferred**). Scores are in-memory only and reset when the window closes, which the README already names as a known gap.

7 What to take away

At this size the layer split looks like over-engineering. It is the opposite. It is the only reason the project’s one substantive claim is checkable at all, and the project knows it: the README’s stated lesson is that “unbeatable” is worth writing down only because an exhaustive enumeration behind it would catch the claim being false, where random simulation never could. The same instinct shows up in miniature throughout, in `WIN_LINES` replacing eight branches with eight tuples, in `ai_mark` being derived instead of stored, and in the refusal to credit alpha-beta for a guarantee it does not provide.

The AI is correct and proven. The window is untested and has at least one real bug in it. Both halves of that sentence are the honest read.

Glossary

Module

One `.py` file. `import game` runs `game.py` once and hands back what it defined.

tkinter

The graphical toolkit that ships inside Python. Windows, buttons, labels, and an event loop. No installation on Windows or macOS; on Debian and Ubuntu it is the separate `python3-tk` system package.

Standard library

Modules arriving with Python itself. This project uses only `tkinter`, `math`, `random` and `unittest`, which is why `requirements.txt` lists no packages, only a comment saying there are none.

Widget

One on-screen thing. Widgets form a tree; the window is that tree drawn.

pack / grid

tkinter's two layout managers. `pack` stacks children in order, `grid` places them in rows and columns. Never mixed inside one parent; here the frames are packed and the nine cells are gridded.

Event loop

`root.mainloop()`. Waits for clicks and timers and calls your functions. While your function runs, the loop is blocked and the window is frozen, which is why `time.sleep` would be the wrong way to pause.

root.after(ms, fn)

Ask the event loop to call `fn` once, later, and return immediately. Not a thread, not a sleep. Returns a cancellable id, which this code discards.

tk.StringVar

A tkinter box holding a string that widgets watch. Both radios in a group share one; clicking writes its value in. Read with `.get()`.

Game tree

Every way the game can unfold. Root is now, branches are legal moves, leaves are finished games.

Recursion

A function calling itself on a smaller version of the problem, stopping at a base case where the answer is obvious. Here the base case is “the game is over, so the score is a fact”.

Minimax

Search the whole game tree; maximise on your turns, minimise on the opponent's. Assumes perfect defence, which is why the result holds against any opponent.

depth

How many moves deep in the search a board is. The board passed to `best_ai_move` is depth 0.

Alpha-beta pruning

Skip branches that cannot change the answer. A pure speedup. Never changes the move chosen.

math.inf

Positive infinity as a float. Seeds the maximiser at `-inf` so the first real score is always an improvement.

Unit test

A function that sets up one situation, runs one piece of code, and asserts the answer. Python's `unittest` module provides the machinery.